Universidad Latina de Costa Rica

Sistemas Operativos II

Título

Proxy HTTP con Caché: Implementación y Rendimiento

Tipo de informe:

Informe para Proyecto Final

Profesor:

Carlos Méndez Rodríguez

Estudiantes:

José Araya Fernández

Amy Morales Cruz

Primer Período Lectivo 2026

Índice

Resumen....2
Introducción....3
Objetivos....4
Objetivo General....5
Objetivos Específicos....5
Marco Teórico....5
Protocolo HTTP....6
Servidores Proxy....6
Caché en Sistemas de Red....6
Sockets y Concurrencia en Go....7
Metodología....7
Resultados....9
1. Arquitectura del Proyecto....10
2. Descripción de los componentes implementados....10
2.1 main.go - Servidor TCP y Gestión de Conexiones....10
2.2 cache.go - Almacenamiento en Memoria con TTL....10
2.3 handler.go - Lógica del Proxy y Política de Caché....11
3. Pruebas Realizadas y Resultados....12
4. Análisis de Concurrencia....12
5. Ajustes Identificados....13
Discusión....14
Conclusión....15
Anexos....16
Bibliografía....17

Resumen

El presente proyecto consistió en el diseño e implementación de un servidor proxy HTTP con caché en el lenguaje de programación Go, utilizando exclusivamente la biblioteca estándar del lenguaje. El sistema fue desarrollado bajo una metodología ágil e iterativa, organizando el trabajo en fases que abarcaron desde la configuración del servidor TCP hasta las pruebas de rendimiento bajo condiciones de concurrencia. La arquitectura resultante se compone de tres módulos con responsabilidades claramente delimitadas: el servidor TCP (main.go), el sistema de caché en memoria con TTL (cache.go) y la lógica del proxy (handler.go). Las pruebas realizadas demostraron que el mecanismo de caché reduce el tiempo de respuesta en aproximadamente un 39%, pasando de 0.018 segundos en el primer acceso a 0.011 segundos en solicitudes posteriores. Asimismo, el modelo de concurrencia basado en goroutines y sync.RWMutex permitió atender múltiples clientes de forma simultánea con un uso eficiente de los recursos del sistema.

Introducción

Los servidores proxy HTTP actúan como intermediarios entre los clientes y los servidores de origen, recibiendo las solicitudes de los usuarios y gestionando su respuesta de manera eficiente. Uno de los mecanismos más importantes que implementan es el almacenamiento en caché, mediante el cual guardan copias locales de los recursos solicitados para poder servirlos directamente en futuras peticiones, reduciendo así el tráfico hacia los servidores de origen y mejorando los tiempos de respuesta (IBM, 2015). Este enfoque resulta especialmente relevante en entornos de alta demanda, donde la optimización del rendimiento y la reducción de la carga en los servidores son aspectos críticos.

El proceso de caché en un proxy HTTP sigue una lógica bien definida: cuando un cliente solicita un recurso, el proxy primero verifica si existe una copia válida en su caché; si la encuentra y está vigente, la entrega directamente al cliente (cache hit); si está desactualizada, la reválida con el servidor de origen; y si no existe, la descarga, la entrega y la almacena para solicitudes futuras (cache miss) (Apache Software Foundation, 2023). La vigencia de los recursos almacenados se determina mediante encabezados HTTP como Cache-Control, Expires y max-age, los cuales permiten tanto al servidor de origen como al administrador del proxy definir políticas precisas de almacenamiento.

El presente proyecto tiene como objetivo diseñar e implementar un proxy HTTP con caché en Go, aplicando conceptos de sockets TCP, concurrencia y manejo de sistemas de archivos. A través de esta implementación se busca analizar el impacto del caché en el rendimiento del sistema, evaluando métricas como el tiempo de respuesta, la tasa de aciertos de caché y el comportamiento bajo carga de múltiples clientes simultáneos.

Objetivos

Objetivo General

Implementar un servidor proxy HTTP con caché en Go que permita reenviar solicitudes a servidores de origen, optimizando el rendimiento mediante el almacenamiento local de recursos frecuentemente solicitados.

Objetivos Específicos

1. Implementar un servidor proxy capaz de recibir y procesar solicitudes HTTP utilizando sockets TCP.
2. Desarrollar un mecanismo de caché que almacene respuestas HTTP para reducir el tiempo de respuesta en solicitudes repetidas.
3. Gestionar la concurrencia mediante hilos para atender múltiples clientes de forma simultánea.

Marco Teórico

Protocolo HTTP

El Protocolo de Transferencia de Hipertexto (HTTP, por sus siglas en inglés) es el fundamento de la comunicación de datos en la World Wide Web. Se trata de un protocolo de capa de aplicación basado en un modelo cliente-servidor, en el cual un cliente envía una solicitud y el servidor devuelve una respuesta (Fielding & Reschke, 2014). HTTP define un conjunto de métodos de solicitud que indican la acción deseada sobre un recurso: GET, para recuperar datos; POST, para enviar información al servidor; PUT, para actualizar un recurso; y DELETE, para eliminarlo, entre otros.

Cada mensaje HTTP está compuesto por una línea de inicio, un conjunto de encabezados (headers) y un cuerpo opcional. Los encabezados transmiten metadatos esenciales como el tipo de contenido, la longitud del mensaje, las directivas de caché y la información de autenticación. Las versiones más utilizadas son HTTP/1.1, que introdujo conexiones persistentes y transferencia por fragmentos, y HTTP/2, que mejoró el rendimiento mediante la multiplexación de solicitudes sobre una sola conexión TCP (Mozilla Developer Network, 2023).

Servidores Proxy

Un servidor proxy es un intermediario que se sitúa entre los clientes y los servidores de destino, recibiendo las solicitudes de los primeros y reenvíandolas en su nombre. Este modelo permite implementar funcionalidades adicionales como control de acceso, registro de actividad, balanceo de carga, compresión de contenido y almacenamiento en caché (Luotonen & Altis, 1994). Existen dos tipos principales: el proxy directo (forward proxy), que actúa en nombre del cliente y es transparente para el servidor; y el proxy inverso (reverse proxy), que actúa en nombre del servidor y es transparente para el cliente.

En el contexto de las redes empresariales y académicas, los proxies directos se utilizan frecuentemente para filtrar contenido, reducir el consumo de ancho de banda y mejorar los tiempos de respuesta mediante la reutilización de recursos previamente descargados. IBM (2015) señala que el proxy guarda copias locales de los recursos solicitados, de modo que las peticiones repetidas pueden ser atendidas directamente desde el almacenamiento local sin necesidad de contactar al servidor de origen.

Caché en Sistemas de Red

La caché web es un mecanismo que permite almacenar temporalmente copias de recursos HTTP con el objetivo de reducir la latencia y el tráfico de red. Cuando un recurso se almacena en caché, las solicitudes posteriores pueden ser atendidas localmente, lo que se denomina cache hit. Si el recurso no está almacenado o ha expirado, se produce un cache miss, y el proxy debe contactar al servidor de origen para obtener una versión actualizada (Apache Software Foundation, 2023).

La validez de los recursos almacenados se controla mediante encabezados HTTP específicos. El encabezado Cache-Control permite definir directivas como max-age, que indica el tiempo máximo en segundos durante el cual un recurso se considera válido; no-cache, que obliga al proxy a revalidar el recurso con el servidor antes de servirlo; y no-store, que prohíbe el almacenamiento en caché. El encabezado Expires cumple una función similar, indicando la fecha y hora exacta en la que el recurso deja de ser válido. Cuando un recurso ha expirado, el proxy puede revalidarlo mediante los encabezados condicionales If-Modified-Since o If-None-Match, evitando la transferencia completa del contenido si no ha cambiado (Apache Software Foundation, 2023).

El almacenamiento en caché puede realizarse en memoria RAM, lo que ofrece tiempos de acceso muy bajos pero capacidad limitada, o en disco, que permite mayor capacidad a costa de una mayor latencia. IBM (2015) indica que los sistemas de proxy mantienen un índice en memoria con información sobre las URLs almacenadas, su ubicación y fecha de expiración, con el fin de acelerar la búsqueda de recursos en caché.

Sockets y Concurrencia en Go

Un socket es un punto de comunicación entre dos procesos a través de una red. En Go, el paquete net de la biblioteca estándar proporciona las abstracciones necesarias para establecer conexiones TCP sin necesidad de gestionar directamente los detalles del protocolo. La función net.Listen permite crear un servidor que escucha conexiones entrantes en un puerto específico, mientras que net.Dial establece una conexión hacia un servidor remoto. Una vez establecida la conexión, ambos extremos intercambian datos mediante la interfaz net.Conn, que expone métodos de lectura y escritura compatibles con el modelo de streams de TCP (Go Authors, s.f.).

Para atender múltiples clientes de forma simultánea, Go utiliza goroutines: funciones ligeras gestionadas por el runtime del lenguaje que permiten ejecutar tareas concurrentes con un costo de memoria significativamente menor al de los hilos del sistema operativo. El patrón habitual consiste en invocar go handleClient(conn) por cada conexión aceptada, delegando su atención a una goroutine independiente sin bloquear el bucle principal del servidor. La comunicación y sincronización entre goroutines se realiza mediante canales (channels), siguiendo la filosofía del lenguaje: "Do not communicate by sharing memory; instead, share memory by communicating" (Go Authors, s.f.). Este modelo resulta especialmente adecuado para un proxy HTTP bajo alta carga, ya que permite gestionar miles de conexiones concurrentes con un uso eficiente de los recursos del sistema.

Metodología

Para el desarrollo del proyecto se adoptó una metodología ágil e iterativa basada en sprints, ampliamente utilizada en proyectos de desarrollo de software por su capacidad de adaptación y entrega continua de valor (Wake Forest University, s.f.). Cada sprint corresponde a un ciclo de trabajo de aproximadamente una semana, al final del cual se obtiene un entregable funcional que sirve de base para la siguiente iteración. Este enfoque permite validar la comunicación entre sockets antes de añadir la complejidad de la caché, reduciendo el riesgo de errores acumulados.

Las fases del proyecto se organizaron de la siguiente manera:

Fase / Sprint	Actividades	Entregable
1. Configuración del servidor	Configuración del ServerSocket y manejo de peticiones mediante hilos (threads).	Servidor capaz de aceptar múltiples conexiones.
2. Implementación del protocolo HTTP	Análisis (parsing) de cabeceras HTTP con soporte para métodos GET y HOST.	Proxy funcional sin caché.
3. Gestión del caché	Implementación de una estructura de datos para almacenamiento local con lógica de expiración.	Sistema de almacenamiento en memoria o disco.
4. Pruebas y ajustes	Pruebas de concurrencia y validación de tiempos de respuesta con herramientas de benchmarking.	Informe de rendimiento y código final.

El manejo de protocolos de red a bajo nivel suele presentar errores inesperados; trabajar por iteraciones permite asegurar que el flujo de datos (Cliente → Proxy → Internet) sea estable antes de intervenir la respuesta para almacenarla. Con un enfoque experimental, se realizarán pruebas comparativas de tiempo de carga con y sin caché para demostrar empíricamente la optimización del rendimiento.

Las herramientas utilizadas para el desarrollo son las siguientes: el lenguaje de programación Go en su versión estable más reciente, empleando el paquete net de la biblioteca estándar para la comunicación TCP y el paquete sync junto con goroutines para la gestión de concurrencia. Como entorno de desarrollo se utilizará WSL2 con Ubuntu 24.04 bajo Windows, que provee un entorno Linux nativo adecuado para el desarrollo y prueba de aplicaciones de red. El proyecto se alojará en un repositorio público de GitHub bajo licencia MIT como parte de los requisitos de entrega, e incluirá un archivo README con instrucciones de configuración y despliegue. Para la distribución y ejecución reproducible del proxy se utilizará Docker, de modo que el sistema pueda desplegarse en cualquier entorno sin dependencias adicionales. Para las pruebas se emplearán cURL y Postman para validar solicitudes HTTP individuales, un

navegador web para pruebas de navegación real, y Apache JMeter para simular múltiples clientes concurrentes y medir el comportamiento del servidor bajo carga (LoadView, s.f.).

Resultados

La implementación del proxy HTTP con caché fue desarrollada en Go, utilizando únicamente la biblioteca estándar del lenguaje. El proyecto se organiza en tres archivos principales: main.go, cache.go y handler.go, cada uno con una responsabilidad bien delimitada. A continuación, se describe en detalle el funcionamiento de cada componente, los resultados de las pruebas realizadas y el análisis del comportamiento concurrente del sistema.

1. Arquitectura del Proyecto

El sistema sigue una arquitectura de tres capas claramente separadas:

- **main.go:** punto de entrada, inicializa el servidor TCP y la caché, y delega cada conexión a una goroutine independiente.
- **cache.go:** estructura de datos en memoria con soporte para TTL (tiempo de vida) y limpieza automática de entradas expiradas.
- **handler.go:** lógica de negocio del proxy; analiza la solicitud HTTP, consulta la caché, y si no hay hit, reenvía la solicitud al servidor de origen y almacena la respuesta.

2. Descripción de los componentes implementados

2.1 main.go - Servidor TCP y Gestión de Conexiones

El archivo main.go constituye el punto de entrada del proxy. Su función es inicializar el directorio de logs, crear la instancia de caché compartida e iniciar el servidor TCP en el puerto 8080. El fragmento central del bucle principal es el siguiente:

```
listener, err := net.Listen("tcp", PORT)
for {
    conn, err := listener.Accept()
    if err != nil {
        log.Println("Error accepting connection:", err)
        continue
    }
    go handleClient(conn, cache)  // goroutine por cliente
}
```

Por cada conexión aceptada se lanza una goroutine (go handleClient), lo que permite atender múltiples clientes de forma simultánea sin bloquear el bucle principal. El uso de defer listener.Close() garantiza que el socket se cierre correctamente cuando el proceso termina.

2.2 cache.go - Almacenamiento en Memoria con TTL

La caché se implementa como un map[string]CacheEntry protegido con un sync.RWMutex, donde la clave es la combinación del método HTTP y la URL de la solicitud (por ejemplo, GET:http://localhost:9090/). Cada entrada almacena el cuerpo de la respuesta (Body []byte) y su tiempo de expiración (ExpiresAt time.Time).

```
type CacheEntry struct {
    Body      []byte
    ExpiresAt time.Time
}

func (c *Cache) Get(key string) ([]byte, bool) {
    c.mu.RLock()
    defer c.mu.RUnlock()
    entry, found := c.entries[key]
    if !found || time.Now().After(entry.ExpiresAt) {
        return nil, false
    }
    return entry.Body, true
}
```

Una goroutine de limpieza (cleanup) se ejecuta en segundo plano con un ticker de 60 segundos, eliminando automáticamente las entradas cuyo tiempo de vida ha expirado. Esto evita un crecimiento indefinido de la caché en memoria.

2.3 handler.go - Lógica del Proxy y Política de Caché

El handler recibe cada conexión de cliente, analiza la solicitud HTTP con http.ReadRequest y aplica la siguiente política de caché: las solicitudes GET se consultan primero en caché (cache hit o cache miss); los demás métodos se reenvían directamente al servidor de origen sin almacenarse (bypass). El TTL configurado es de 30 segundos (CACHE_TTL = 30 * time.Second).

```
cacheKey := req.Method + ":" + req.URL.String()

if req.Method == http.MethodGet {
    if cached, found := cache.Get(cacheKey); found {
        fmt.Printf("[CACHE HIT]  %s %s\n", req.Method, req.URL)
        clientConn.Write(cached)
        return
    }
    fmt.Printf("[CACHE MISS] %s %s\n", req.Method, req.URL)
} else {
    fmt.Printf("[BYPASS]     %s %s\n", req.Method, req.URL)
```

```
}
```

Cuando se produce un cache miss, el handler conecta al servidor de origen mediante net.Dial, reenvía la solicitud, y utiliza io.TeeReader para leer la respuesta mientras la copia simultáneamente al cliente y al buffer de caché. Esto evita leer la respuesta dos veces y minimiza la latencia adicional introducida por el proxy.

3. Pruebas Realizadas y Resultados

Las pruebas se realizaron en un entorno WSL2 con Ubuntu 24.04 y Go 1.22.2. Se levantó un servidor HTTP local con Python (python3 -m http.server 9090) para simular el servidor de origen, y se utilizó curl con el flag --proxy para redirigir las solicitudes al proxy.

La secuencia de prueba fue la siguiente: se envió la misma solicitud GET dos veces consecutivas a través del proxy. La primera solicitud generó un CACHE MISS, lo que implicó una conexión real al servidor de origen. La segunda solicitud fue respondida directamente desde la caché, generando un CACHE HIT sin contactar al servidor de origen.

Tabla 1.

Comparación de Tiempos: CACHE MISS vs CACHE HIT

Solicitud	Tipo	Tiempo real (s)	Velocidad (KB/s)
1ra (GET /)	CACHE MISS	0.018	128k
2da (GET /)	CACHE HIT	0.011	553k → 771k

Los resultados muestran que la segunda solicitud fue notablemente más rápida: el tiempo real disminuyó de 0.018s a 0.011s (una mejora de aproximadamente 39%), y la velocidad de transferencia aumentó de 128 KB/s a 771 KB/s. Esto confirma que el caché reduce efectivamente la latencia al eliminar la conexión de red al servidor de origen.

Cabe destacar que durante las pruebas con servidores externos (como http://example.com), el proxy recibió correctamente la solicitud y registró el CACHE MISS, pero devolvió un error HTTP 502 Bad Gateway. Esto se debió a restricciones de red del entorno WSL2, no a un fallo en la lógica del proxy.

4. Análisis de Concurrencia

Uno de los requisitos centrales del proyecto era el manejo concurrente de múltiples clientes. El diseño implementado utiliza el modelo de concurrencia nativo de Go, basado en goroutines y mutexes, lo que permite atender simultáneamente un número elevado de conexiones con un consumo de memoria muy bajo.

Tabla 2.

Mecanismos de Concurrencia Implementados

Mecanismo	Comportamiento	Protección de datos
goroutine por cliente	go handleClient(conn, cache)	sync.RWMutex en caché
Lectura concurrente	RLock() / RUnlock()	Múltiples lectores simultáneos
Escritura exclusiva	Lock() / Unlock()	Un escritor a la vez
Limpieza automática	ticker cada 60s en goroutine	Evita entradas expiradas

El uso de sync.RWMutex es particularmente relevante para la caché: permite que múltiples goroutines lean de forma simultánea (RLock), pero garantiza exclusividad cuando una goroutine escribe o elimina una entrada (Lock). Esto maximiza el rendimiento en escenarios de alta concurrencia donde la mayoría de operaciones son lecturas (cache hits).

El modelo seguido es el patrón estándar de Go para servidores de red: el bucle principal acepta conexiones y delega inmediatamente cada una a una goroutine independiente mediante go handleClient(conn, cache). Esto asegura que una conexión lenta o bloqueada no afecte al resto de los clientes.

5. Ajustes Identificados

Con base en los resultados obtenidos, se identificaron los siguientes aspectos de mejora para futuras iteraciones:

- Incrementar el TTL de caché más allá de los 30 segundos actuales para recursos estáticos, lo que aumentaría la tasa de cache hit en pruebas más prolongadas.
- Implementar soporte para el método CONNECT de HTTP, necesario para el manejo de tráfico HTTPS a través de un proxy de túnel.
- Añadir un límite de tamaño máximo de caché para evitar el consumo excesivo de memoria bajo carga sostenida.
- Incorporar métricas de rendimiento persistentes (total de hits, misses y ratio de aciertos) para facilitar el análisis cuantitativo en pruebas con Apache JMeter.

Discusión

Los resultados obtenidos confirman que la implementación del caché tiene un impacto directo y medible en el rendimiento del proxy. La reducción del 39% en el tiempo de respuesta entre el primer acceso (CACHE MISS) y los accesos posteriores (CACHE HIT) evidencia que eliminar la conexión de red al servidor de origen es el factor determinante en la mejora de latencia. Este comportamiento es consistente con lo descrito en la literatura: IBM (2015) señala que el proxy mantiene copias locales de los recursos para atender peticiones repetidas sin contactar al servidor de origen.

En cuanto a la concurrencia, el uso de goroutines resultó adecuado para el alcance del proyecto. A diferencia de los hilos del sistema operativo, las goroutines tienen un costo de memoria significativamente menor, lo que permite escalar a un mayor número de conexiones simultáneas sin degradar el rendimiento. El uso de sync.RWMutex fue una decisión de diseño acertada: al permitir múltiples lecturas concurrentes y bloquear únicamente durante las escrituras, se maximiza el rendimiento en escenarios donde la mayoría de las operaciones son CACHE HITs.

Sin embargo, se identificaron algunas limitaciones durante el desarrollo. El TTL fijo de 30 segundos resultó adecuado para las pruebas realizadas, pero podría ser insuficiente en escenarios reales con recursos estáticos de larga duración. Asimismo, la ausencia de soporte para el método CONNECT limita el uso del proxy con tráfico HTTPS. Por otra parte, las restricciones de red del entorno WSL2 impidieron realizar pruebas con servidores externos reales, lo que acotó el alcance de las métricas obtenidas a un entorno local controlado. Estas limitaciones no invalidan los resultados, pero sí representan áreas de mejora para iteraciones futuras, tal como se detalló en la sección de ajustes identificados.

Conclusión

El proyecto logró cumplir los tres objetivos específicos planteados. Se implementó un servidor proxy funcional capaz de recibir y procesar solicitudes HTTP mediante sockets TCP, se desarrolló un mecanismo de caché en memoria que demostró empíricamente reducir los tiempos de respuesta en solicitudes repetidas, y se gestionó la concurrencia mediante goroutines para atender múltiples clientes de forma simultánea. En conjunto, estos componentes conforman un sistema coherente que refleja los conceptos centrales del curso: sockets, concurrencia y gestión de recursos del sistema operativo.

Más allá de los resultados técnicos, el desarrollo del proyecto permitió comprender de forma práctica cómo los sistemas operativos subyacen en el funcionamiento de herramientas de red reales. La gestión de goroutines, la sincronización con mutexes y el manejo de conexiones TCP son abstracciones que Go expone de manera accesible, pero que internamente dependen de los mecanismos de planificación y gestión de memoria del sistema operativo. Como trabajo futuro, se propone implementar soporte para HTTPS mediante el método CONNECT, incorporar un límite de tamaño máximo de caché y añadir métricas persistentes de rendimiento que permitan un análisis más exhaustivo bajo carga sostenida.

Anexos

Tabla 1.

Comparación de Tiempos: CACHE MISS vs CACHE HIT

Solicitud	Tipo	Tiempo real (s)	Velocidad (KB/s)
1ra (GET /)	CACHE MISS	0.018	128k
2da (GET /)	CACHE HIT	0.011	553k → 771k

Tabla 2.

Mecanismos de Concurrencia Implementados

Mecanismo	Comportamiento	Protección de datos
goroutine por cliente	go handleClient(conn, cache)	sync.RWMutex en caché
Lectura concurrente	RLock() / RUnlock()	Múltiples lectores simultáneos
Escritura exclusiva	Lock() / Unlock()	Un escritor a la vez
Limpieza automática	ticker cada 60s en goroutine	Evita entradas expiradas

Bibliografía

- Apache Software Foundation. (s.f.). *HTTP Proxy Caching*. Apache Traffic Server. https://docs.trafficserver.apache.org/admin-guide/configuration/cache-basics.en.html
- IBM. (2026). *Overview of proxy server caching*. IBM WebSphere Application Server Network Deployment 8.5.5. https://www.ibm.com/docs/en/was-nd/8.5.5?topic=caching-overview-proxy-server
- LoadView. (s. f.). *Pruebas de carga: Las 10 mejores herramientas de código abierto y de pago*. https://www.loadview-testing.com/es/blog/pruebas-de-carga-revisadas-las-10-mejores-herramientas-de-prueba-de-carga-de-codigo-abierto-y-de-pago/
- Wake Forest University. (s. f.). *Benefits of agile project management*. School of Professional Studies. https://sps.wfu.edu/articles/benefits-agile-project-management/
- Fielding, R., & Reschke, J. (2014). *Hypertext Transfer Protocol (HTTP/1.1): Message syntax and routing (RFC 7230).* Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc7230
- Luotonen, A., & Altis, K. (2003). *World-Wide Web proxies.* Computer Networks and ISDN Systems, 27(2), 147-154. https://doi.org/10.1016/0169-7552(94)90128-7
- Mozilla Developer Network. (2023). *HTTP*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/HTTP
- Go Authors. (s.f.). *The Go programming language specification — Package net*. Google. https://pkg.go.dev/net